



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Yash Vilas Karande
Roll No. :	17
Experiment No.:	3
Title of the Experiment:	Implement Correlation Analysis, Distance Measures, and Data Preprocessing Techniques
Date of Performance:	29/08/26
Date of Submission:	05/08/26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Tatwadarshi P.N.**

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To implement correlation analysis, distance measures, and basic data preprocessing techniques on the Pima Indians Diabetes Dataset to identify relationships between variables and prepare data for further analysis.

Objectives

After completing this experiment, the student will be able to:

1. Apply correlation and distance measures to analyze relationships and similarities among data observations.
2. Implement basic data preprocessing techniques to prepare a dataset for statistical analysis and machine learning.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical diagnostic information for **768 female patients** and is used to study factors associated with diabetes.

The dataset contains numerical attributes and a binary outcome variable indicating whether the patient is diabetic.

Procedure

1. Load the Pima Indians Diabetes Dataset.



2. Identify missing or invalid values, particularly zero values in medical attributes where zero may be unrealistic.
3. Handle selected missing values using an appropriate imputation technique.
4. Calculate the correlation matrix for numerical variables.
5. Visualize correlations using a heatmap.
6. Calculate Euclidean, Manhattan, and Cosine distances for selected observations.
7. Apply an appropriate normalization or standardization technique.
8. Compare the data before and after preprocessing.
9. Interpret the obtained correlation and distance measures.

Description of the Experiment

Correlation analysis helps determine the strength and direction of relationships between variables.

For example, the relationship between:

Glucose and BMI

can be analyzed using correlation.

Distance measures determine the similarity or dissimilarity between observations.

Data preprocessing is performed to improve the quality and consistency of data before further statistical or machine learning analysis.

The experiment follows: Raw Dataset, Identify Invalid/Missing Values, Data Preprocessing, Correlation Analysis, Distance Calculation and Interpretation

Detailed Description of Statistical Methods

Correlation

Correlation measures the strength and direction of the linear relationship between two variables.

The Pearson correlation coefficient is:



$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

The value of r lies between:

$$-1 \leq r \leq 1$$

$r \approx 1$: Strong positive relationship

$r \approx -1$: Strong negative relationship

$r \approx 0$: Weak or no linear relationship

A correlation heatmap can be used to visualize relationships among multiple variables.

Euclidean Distance

Euclidean distance between two observations is:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

It represents the straight-line distance between two points.

Manhattan Distance

Manhattan distance is:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

It measures distance by summing the absolute differences between corresponding features.

Cosine Distance

Cosine similarity is:

$$\cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|}$$

Cosine distance is:

$$d_{\cosine} = 1 - \cos(\theta)$$

It measures the difference in the direction of two vectors.

Data Preprocessing

Missing Value Handling

Invalid or missing values may be replaced using techniques such as:

- Mean imputation
- Median imputation

For medical variables, median imputation may be preferred when extreme values are present.



Normalization

Min-Max normalization transforms values into a specified range, usually [1]:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Standardization

Standardization transforms a variable to have mean 0 and standard deviation 1:

$$Z = \frac{x - \mu}{\sigma}$$

Scaling is important when distance-based methods are used because variables with larger numerical ranges can otherwise dominate the distance calculation.

Implementation

```
[1] 3s
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics.pairwise import euclidean_distances, manhattan_distances, cosine_distances

[2] 0s
df = pd.read_csv("diabetes.csv")
```

```
[3] 0s
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

Next steps: [Generate code with df](#) [New interactive sheet](#)



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
[4] df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Pregnancies           768 non-null   int64  
1   Glucose               768 non-null   int64  
2   BloodPressure         768 non-null   float64
3   SkinThickness         768 non-null   int64  
4   Insulin               768 non-null   float64
5   BMI                  768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                  768 non-null   int64  
8   Outcome              768 non-null   int64  
dtypes: float64(4), int64(5)
memory usage: 54.1 KB
```

```
[5] df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.677083	72.389323	29.089844	141.753906	32.434635	0.471876	33.240885	0.348958
std	3.369578	30.464161	12.106039	8.890820	89.100847	6.880498	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	25.000000	102.500000	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	28.000000	102.500000	32.050000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	169.500000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

```
[6] (df == 0).sum()
Pregnancies    111
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
DiabetesPedigreeFunction 0
Age              0
Outcome          500
dtype: int64
```

```
[7] cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
df[cols] = df[cols].replace(0, np.nan)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
[8] df.isnull().sum()
0s
Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64
```

```
[9] imputer = SimpleImputer(strategy='median')
df[cols] = imputer.fit_transform(df[cols])
```

```
[10] df.isnull().sum()
0s
Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64
```

```
[11] corr = df.corr()
corr
Pregnancies Glucose BloodPressure SkinThickness Insulin BMI DiabetesPedigreeFunction Age Outcome
Pregnancies 1.000000 0.130155 0.209151 0.089028 0.058767 0.023890 -0.033523 0.544341 0.221898
Glucose 0.130155 1.000000 0.225141 0.229289 0.490015 0.236171 0.138353 0.268910 0.495990
BloodPressure 0.209151 0.225141 1.000000 0.199349 0.070128 0.286399 -0.001443 0.325135 0.174469
SkinThickness 0.089028 0.229289 0.199349 1.000000 0.200129 0.566086 0.106280 0.129537 0.295138
Insulin 0.058767 0.490015 0.070128 0.200129 1.000000 0.238443 0.146878 0.123629 0.377081
BMI 0.023890 0.236171 0.286399 0.566086 0.238443 1.000000 0.152771 0.027849 0.315577
DiabetesPedigreeFunction -0.033523 0.138353 -0.001443 0.106280 0.146878 0.152771 1.000000 0.033561 0.173844
Age 0.544341 0.268910 0.325135 0.129537 0.123629 0.027849 0.033561 1.000000 0.238356
Outcome 0.221898 0.495990 0.174469 0.295138 0.377081 0.315577 0.173844 0.238356 1.000000
```



Vidyavardhini's College of Engineering and Technology

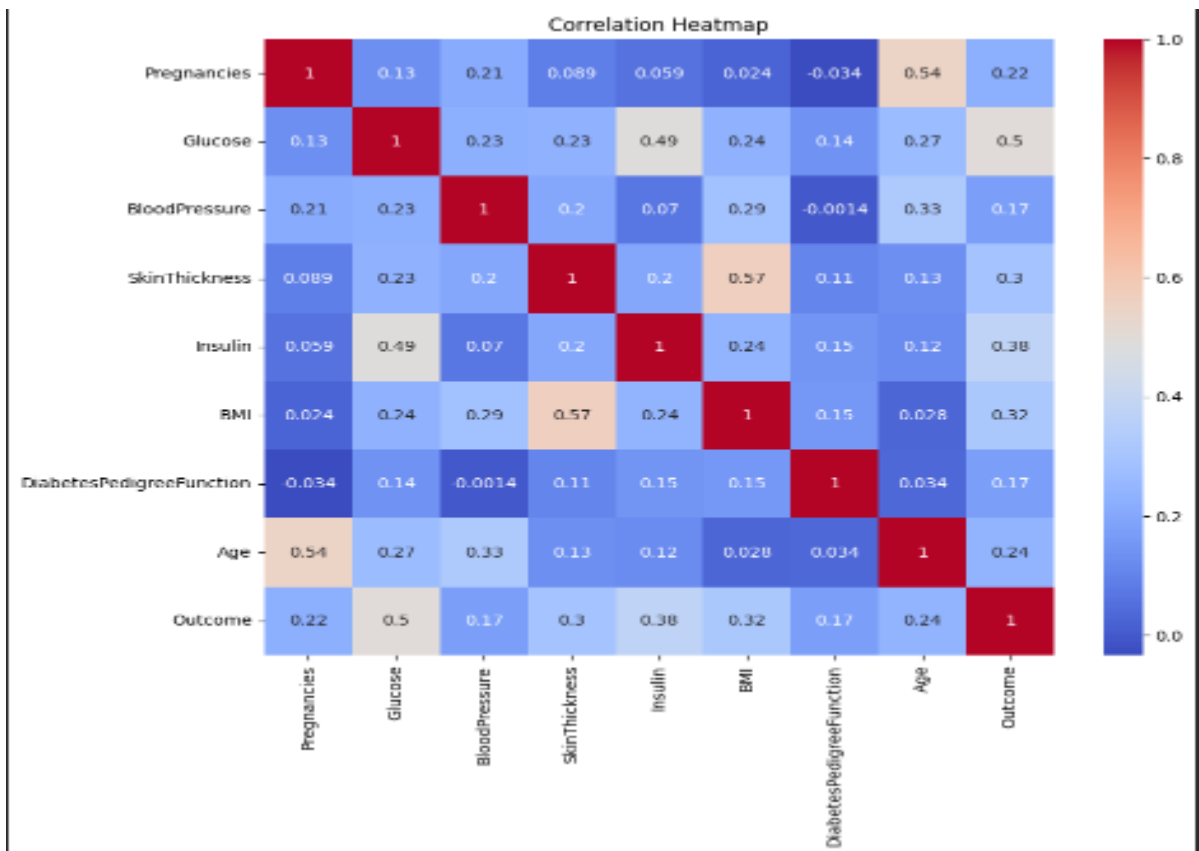
Department of Artificial Intelligence & Data Science

```
[12]
✓ 0s plt.figure(figsize=(10,8))

sns.heatmap(corr,
             annot=True,
             cmap='coolwarm')

plt.title("Correlation Heatmap")

plt.show()
```



```
[13]
✓ 0s x = df.iloc[[0]]
     y = df.iloc[[1]]

[14]
✓ 0s euclidean_distances(x, y)

array([[94.68936675]])

[15]
✓ 0s manhattan_distances(x, y)

... array([[174.276]])

[16]
✓ 0s cosine_distances(x, y)

array([[0.01223767]])
```

+ Code

+ Text



```
[17] ✓ Os
scaler = MinMaxScaler()

normalized = scaler.fit_transform(df)

normalized_df = pd.DataFrame(normalized, columns=df.columns)

normalized_df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	0.352941	0.670968	0.489796	0.304348	0.186899	0.314928	0.234415	0.483333	1.0
1	0.058824	0.264516	0.428571	0.239130	0.106370	0.171779	0.116567	0.166667	0.0
2	0.470588	0.896774	0.408163	0.271739	0.186899	0.104294	0.253629	0.183333	1.0
3	0.058824	0.290323	0.428571	0.173913	0.096154	0.202454	0.038002	0.000000	0.0
4	0.000000	0.600000	0.163265	0.304348	0.185096	0.509202	0.943638	0.200000	1.0

```
[18] ✓ Os
scaler = StandardScaler()

standardized = scaler.fit_transform(df)

standardized_df = pd.DataFrame(standardized, columns=df.columns)

standardized_df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	0.639947	0.864625	-0.032180	0.665181	0.311604	0.169483	0.468492	1.425995	1.365896
1	-0.844885	-1.204727	-0.528124	-0.010112	-0.440843	-0.848549	-0.365061	-0.190672	-0.732120
2	1.233880	2.014265	-0.693438	0.327535	0.311604	-1.328478	0.604397	-0.105584	1.365896
3	-0.844885	-1.073339	-0.528124	-0.685405	-0.536303	-0.630399	-0.920763	-1.041549	-0.732120
4	-1.141852	0.503310	-2.677212	0.665181	0.294758	1.551096	5.484909	-0.020496	1.365896

Questions to be Answered by Students

1. Which pair of variables has the strongest positive correlation in the dataset?

ANS: Strongest positive correlation:

- Age and Pregnancies generally show one of the highest positive correlations.

Reason:

As age increases, the number of pregnancies tends to increase because older patients have had more time to experience pregnancies.

Another important positive relationship is:

- Glucose and Outcome

This indicates that higher glucose levels are associated with a higher probability of diabetes.

2. Which pair of variables has the strongest negative correlation, if any?

ANS: The dataset does not contain a strong negative correlation.

The most negative correlations are usually very weak, close to zero.



Examples of weak negative relationships may occur between:

3. BloodPressure and Outcome
4. SkinThickness and some other variables

However, their correlation values are small, meaning there is no significant inverse relationship among the variables.

- Conclusion:

No pair of variables shows a strong negative correlation in this dataset.

3. How do the Euclidean, Manhattan, and Cosine distances differ for the selected observations?

ANS: For two selected patient observations:

- Euclidean Distance

Measures the straight-line distance between two patients.

It considers the squared difference of every feature.

Larger values indicate greater overall difference.

Example interpretation:

If two patients have very different glucose, BMI, age, and insulin values, Euclidean distance becomes large.

- Manhattan Distance

Measures the total absolute difference between features.

It adds the individual differences instead of squaring them.

It is less affected by extreme differences compared to Euclidean distance.

Example:

It measures how much total change is needed to move from one patient's measurements to another.

- Cosine Distance

Measures the difference in direction between two data vectors.

It focuses on the pattern of values rather than their actual magnitude.

A smaller cosine distance means the patients have similar feature patterns.



4. Which preprocessing technique was applied to the invalid or missing values, and why was it selected?

ANS: The invalid zero values in medical attributes such as:

2. Glucose
3. BloodPressure
4. SkinThickness
5. Insulin
6. BMI

were replaced with missing values (NaN).

The missing values were handled using:

Median Imputation

`SimpleImputer(strategy='median')`

Why median imputation was selected?

Medical datasets often contain extreme values (outliers).

The median is not affected significantly by extreme values.

It provides a more reliable replacement compared to mean imputation.

Example:

If insulin values contain a few extremely high readings, the mean may become misleading, while the median remains stable.

5. How does normalization or standardization affect the calculated distance between observations?

ANS: Before preprocessing, variables have different ranges.

Example:

Age: 21–80

Glucose: 50–200

Insulin: 0–850



Because insulin has a much larger range, it can dominate distance calculations.

After Normalization:

Values are converted to a common range (usually 0 to 1).

Effect:

All features contribute equally.

Large-scale variables no longer dominate.

Distance values become more balanced.

After Standardization:

Values are transformed so that:

Mean = 0

Standard deviation = 1

Effect:

Features are centered around the same scale.

Distance calculations become more meaningful, especially for machine learning algorithms.

Conclusion

Correlation analysis, distance measures, and data preprocessing techniques were implemented on the Pima Indians Diabetes Dataset. Correlation analysis helped identify relationships between medical variables, while distance measures quantified the similarity or dissimilarity between observations. Data preprocessing techniques helped improve the consistency and suitability of the data for further analysis and machine learning applications.